Open in Colab

Prelab 10.2 Machine Learning 3 - Real-time Sound Recognition for Classification

CNN model training and evaluation

In [23]:
```
# Let's check the installed tensorflow version
import tensorflow as tf

print("TensorFlow Version is", tf.__version__)
```

```
TensorFlow Version is 2.19.0
```

In [24]:
```
# Install required Python packages for this colab
!pip install librosa
!pip install matplotlib
!pip install seaborn
```

```
Requirement already satisfied: librosa in /usr/local/lib/python3.12/dist-packages (0.11.0)
Requirement already satisfied: audioread>=2.1.9 in /usr/local/lib/python3.12/dist-packages (from librosa) (3.
1.0)
Requirement already satisfied: numba>=0.51.0 in /usr/local/lib/python3.12/dist-packages (from librosa) (0.60.
0)
Requirement already satisfied: numpy>=1.22.3 in /usr/local/lib/python3.12/dist-packages (from librosa) (2.0.
2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from librosa) (1.16.
3)
Requirement already satisfied: scikit-learn>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from librosa)
(1.6.1)
Requirement already satisfied: joblib>=1.0 in /usr/local/lib/python3.12/dist-packages (from librosa) (1.5.3)
Requirement already satisfied: decorator>=4.3.0 in /usr/local/lib/python3.12/dist-packages (from librosa) (4.
4.2)
Requirement already satisfied: soundfile>=0.12.1 in /usr/local/lib/python3.12/dist-packages (from librosa)
(0.13.1)
Requirement already satisfied: pooch>=1.1 in /usr/local/lib/python3.12/dist-packages (from librosa) (1.9.0)
Requirement already satisfied: soxr>=0.3.2 in /usr/local/lib/python3.12/dist-packages (from librosa) (1.0.0)
Requirement already satisfied: typing_extensions>=4.1.1 in /usr/local/lib/python3.12/dist-packages (from libr
osa) (4.15.0)
Requirement already satisfied: lazy_loader>=0.1 in /usr/local/lib/python3.12/dist-packages (from librosa) (0.
5)
Requirement already satisfied: msgpack>=1.0 in /usr/local/lib/python3.12/dist-packages (from librosa) (1.1.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from lazy_loader>=0.1->l
ibrosa) (26.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from nu
mba>=0.51.0->librosa) (0.43.0)
Requirement already satisfied: platformdirs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from pooch>=1.
1->librosa) (4.9.4)
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.12/dist-packages (from pooch>=1.1->
librosa) (2.32.4)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-l
earn>=1.1.0->librosa) (3.6.0)
Requirement already satisfied: cffi>=1.0 in /usr/local/lib/python3.12/dist-packages (from soundfile>=0.12.1->
librosa) (2.0.0)
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages (from cffi>=1.0->soundfil
e>=0.12.1->librosa) (3.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requ
ests>=2.19.0->pooch>=1.1->librosa) (3.4.6)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0
->pooch>=1.1->librosa) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=
2.19.0->pooch>=1.1->librosa) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=
2.19.0->pooch>=1.1->librosa) (2026.2.25)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib)
(1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.1
2.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib)
(4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib)
(1.5.0)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.0.
2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib)
(26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.
0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib)
(3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotl
ib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7
->matplotlib) (1.17.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /usr/local/lib/python3.12/dist-packages (from seaborn)
(2.0.2)
Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.12/dist-packages (from seaborn) (2.2.2)
```

```
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /usr/local/lib/python3.12/dist-packages (from seabo
rn) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=
3.6.1,>=3.4->seaborn) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.
1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib!
=3.6.1,>=3.4->seaborn) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!
=3.6.1,>=3.4->seaborn) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=
3.6.1,>=3.4->seaborn) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>
=3.4->seaborn) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=
3.6.1,>=3.4->seaborn) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotl
ib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.2->sea
born) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.2->s
eaborn) (2025.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7
->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
```

2.1 Data loading and signal processing

Before we load data and analyze them, let's figure out the experiments for the training data collection. As discussed in the previous session (Prelab 10.1), we defined the machine's three states: (1) OFF, (2), ON in vacuuming, and (3) ON in air-leaking. The data collection was basically performed in the lab space in a quiet environment. Data was collected from the MTConnect sound stream in WAV format. The audio specifications, such as sampling rate, resolution, and so on, were the same as the MTConnect stream. The configuration of data collection is illustrated in Figure 7. To make a noisy environment, a loudspeaker was placed near the pump.

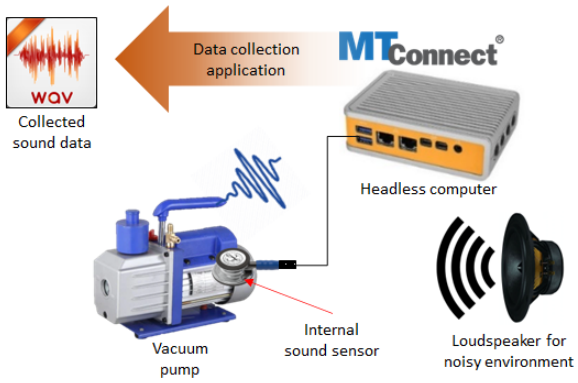


Figure 7 System configuration for sound data collection

Sound signals from the internal sound sensor for each state of the machine were collected for 1 minute. On top of that, simulating other environmental conditions, the same amount of data was collected in a loud environment in which a loudspeaker generated white noise. Therefore, we have three states data and total 6 sound data in Github repo. You will figure out the condition in the filename. Note that each data is 1 minute-long. Description for the filename and states is below.

- **OFF.wav**
  - Machine is turned off, state 1 (OFF)
- **OFF_with_noise.wav**
  - Machine is turned off, state 1 (OFF)
- **ON_vacuum.wav**
  - Machine is on in vacuuming, state 2 (ON/Vacuuming)
- **ON_vacuum_with_noise.wav**

    - Machine is on in vacuuming and noisy environment, state 2 (ON/Vacuuming)
- **ON_air_leaking.wav**
    - Machine is on in air-leaking, state 3 (ON/Air-leaking)
- **ON_air_leaking_with_noise.wav**
    - Machine is on in air-leaking and noisy environment, state 3 (ON/Air-leaking)

Let's listen to the sound of each case and then load data to analyze it.

Loading data

We will use bash command in the code block below to load data in Colab session. Note that you can use Colab interface to upload data or mount your Google Drive instead.

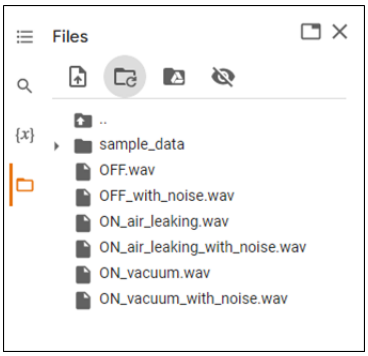
In [25]:
```
# loading sound data in this Colab session by bash
# It takes a few seconds.
!wget -O OFF.wav https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/lab10/prelab_data/OFF.wav?raw
!wget -O OFF_with_noise.wav https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/lab10/prelab_data/
!wget -O ON_vacuum.wav https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/lab10/prelab_data/ON_va
!wget -O ON_vacuum_with_noise.wav https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/lab10/prelab
!wget -O ON_air_leaking.wav https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/lab10/prelab_data/
!wget -O ON_air_leaking_with_noise.wav https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/lab10/p
```

```
--2026-04-09 00:46:56--  https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/lab10/prelab_data/OFF.wav?raw=true
Resolving github.com (github.com)... 140.82.113.3
Connecting to github.com (github.com)|140.82.113.3|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://github.com/purduelamm/purdue_me597_iiot/raw/refs/heads/main/lab/lab10/prelab_data/OFF.wav [following]
--2026-04-09 00:46:56--  https://github.com/purduelamm/purdue_me597_iiot/raw/refs/heads/main/lab/lab10/prelab_data/OFF.wav
Reusing existing connection to github.com:443.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/refs/heads/main/lab/lab10/prelab_data/OFF.wav [following]
--2026-04-09 00:46:57--  https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/refs/heads/main/lab/lab10/prelab_data/OFF.wav
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 5760044 (5.5M) [audio/wav]
Saving to: ‘OFF.wav’

OFF.wav             100%[===================>]   5.49M  --.-KB/s    in 0.06s

2026-04-09 00:46:57 (89.2 MB/s) - ‘OFF.wav’ saved [5760044/5760044]

--2026-04-09 00:46:57--  https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/lab10/prelab_data/OFF_with_noise.wav?raw=true
Resolving github.com (github.com)... 140.82.113.4
Connecting to github.com (github.com)|140.82.113.4|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://github.com/purduelamm/purdue_me597_iiot/raw/refs/heads/main/lab/lab10/prelab_data/OFF_with_noise.wav [following]
--2026-04-09 00:46:58--  https://github.com/purduelamm/purdue_me597_iiot/raw/refs/heads/main/lab/lab10/prelab_data/OFF_with_noise.wav
Reusing existing connection to github.com:443.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/refs/heads/main/lab/lab10/prelab_data/OFF_with_noise.wav [following]
--2026-04-09 00:46:58--  https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/refs/heads/main/lab/lab10/prelab_data/OFF_with_noise.wav
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.109.133, 185.199.111.133, 185.199.108.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.109.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 5759020 (5.5M) [audio/wav]
Saving to: ‘OFF_with_noise.wav’

OFF_with_noise.wav  100%[===================>]   5.49M  --.-KB/s    in 0.05s

2026-04-09 00:46:59 (117 MB/s) - ‘OFF_with_noise.wav’ saved [5759020/5759020]

--2026-04-09 00:46:59--  https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/lab10/prelab_data/ON_vacuum.wav?raw=true
Resolving github.com (github.com)... 140.82.113.4
Connecting to github.com (github.com)|140.82.113.4|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://github.com/purduelamm/purdue_me597_iiot/raw/refs/heads/main/lab/lab10/prelab_data/ON_vacuum.wav [following]
--2026-04-09 00:46:59--  https://github.com/purduelamm/purdue_me597_iiot/raw/refs/heads/main/lab/lab10/prelab_data/ON_vacuum.wav
Reusing existing connection to github.com:443.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/refs/heads/main/lab/lab10/prelab_data/ON_vacuum.wav [following]
--2026-04-09 00:47:00--  https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/refs/heads/main/lab/lab10/prelab_data/ON_vacuum.wav
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.109.133, 185.199.111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443... connected.
```

```
HTTP request sent, awaiting response... 200 OK
Length: 5760044 (5.5M) [audio/wav]
Saving to: ‘ON_vacuum.wav’

ON_vacuum.wav       100%[===================>]   5.49M  --.-KB/s    in 0.06s

2026-04-09 00:47:00 (92.8 MB/s) - ‘ON_vacuum.wav’ saved [5760044/5760044]

--2026-04-09 00:47:00--  https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/lab10/prelab_data/ON_v
acuum_with_noise.wav?raw=true
Resolving github.com (github.com)... 140.82.113.4
Connecting to github.com (github.com)|140.82.113.4|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://github.com/purduelamm/purdue_me597_iiot/raw/refs/heads/main/lab/lab10/prelab_data/ON_vacuum
_with_noise.wav [following]
--2026-04-09 00:47:01--  https://github.com/purduelamm/purdue_me597_iiot/raw/refs/heads/main/lab/lab10/prelab
_data/ON_vacuum_with_noise.wav
Reusing existing connection to github.com:443.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/refs/heads/main/lab/lab10/prelab_dat
a/ON_vacuum_with_noise.wav [following]
--2026-04-09 00:47:01--  https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/refs/heads/main/lab/l
ab10/prelab_data/ON_vacuum_with_noise.wav
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.109.133, 185.199.
111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 5759020 (5.5M) [audio/wav]
Saving to: ‘ON_vacuum_with_noise.wav’

ON_vacuum_with_nois 100%[===================>]   5.49M  --.-KB/s    in 0.05s

2026-04-09 00:47:02 (101 MB/s) - ‘ON_vacuum_with_noise.wav’ saved [5759020/5759020]

--2026-04-09 00:47:02--  https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/lab10/prelab_data/ON_a
ir_leaking.wav?raw=true
Resolving github.com (github.com)... 140.82.113.4
Connecting to github.com (github.com)|140.82.113.4|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://github.com/purduelamm/purdue_me597_iiot/raw/refs/heads/main/lab/lab10/prelab_data/ON_air_le
aking.wav [following]
--2026-04-09 00:47:02--  https://github.com/purduelamm/purdue_me597_iiot/raw/refs/heads/main/lab/lab10/prelab
_data/ON_air_leaking.wav
Reusing existing connection to github.com:443.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/refs/heads/main/lab/lab10/prelab_dat
a/ON_air_leaking.wav [following]
--2026-04-09 00:47:03--  https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/refs/heads/main/lab/l
ab10/prelab_data/ON_air_leaking.wav
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.109.133, 185.199.
111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 5760044 (5.5M) [audio/wav]
Saving to: ‘ON_air_leaking.wav’

ON_air_leaking.wav  100%[===================>]   5.49M  --.-KB/s    in 0.05s

2026-04-09 00:47:03 (109 MB/s) - ‘ON_air_leaking.wav’ saved [5760044/5760044]

--2026-04-09 00:47:03--  https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/lab10/prelab_data/ON_a
ir_leaking_with_noise.wav?raw=true
Resolving github.com (github.com)... 140.82.113.4
Connecting to github.com (github.com)|140.82.113.4|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://github.com/purduelamm/purdue_me597_iiot/raw/refs/heads/main/lab/lab10/prelab_data/ON_air_le
aking_with_noise.wav [following]
--2026-04-09 00:47:04--  https://github.com/purduelamm/purdue_me597_iiot/raw/refs/heads/main/lab/lab10/prelab
_data/ON_air_leaking_with_noise.wav
Reusing existing connection to github.com:443.
```

```
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/refs/heads/main/lab/lab10/prelab_data/ON_air_leaking_with_noise.wav [following]
--2026-04-09 00:47:04--  https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/refs/heads/main/lab/lab10/prelab_data/ON_air_leaking_with_noise.wav
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 5759020 (5.5M) [audio/wav]
Saving to: 'ON_air_leaking_with_noise.wav'

ON_air_leaking_with 100%[===================>]   5.49M  --.-KB/s    in 0.05s

2026-04-09 00:47:05 (105 MB/s) - 'ON_air_leaking_with_noise.wav' saved [5759020/5759020]
```

After running the cell above, Files of your Colab session look like the capture below.

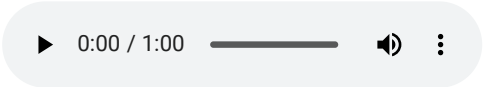


Let's play each data in the following code blocks and perform Task 2.1.

In [26]:
```python
from IPython.display import Audio, display

# Filenames for each data
OFF = "OFF.wav"  # OFF state filename
OFF_with_noise = "OFF_with_noise.wav"  # OFF in noisy environment
ON_vacuum = "ON_vacuum.wav"  # ON in vacuuming
ON_vacuum_with_noise = (
    "ON_vacuum_with_noise.wav"  # ON in vacuuming and noisy environment
)
ON_air_leaking = "ON_air_leaking.wav"  # ON in air-leaking
ON_air_leaking_with_noise = (
    "ON_air_leaking_with_noise.wav"  # On in air-leaking and noisy environment
)
```

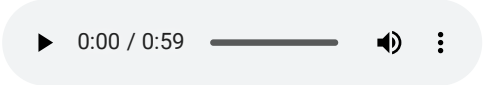
In [27]:
```python
print(OFF)
display(Audio(OFF))
```

```
OFF.wav
```



In [28]:
```python
print(OFF_with_noise)
display(Audio(OFF_with_noise))
```

```
OFF_with_noise.wav
```



In [29]:
```python
print(ON_vacuum)
display(Audio(ON_vacuum))
```

```
ON_vacuum.wav
```

0:00 / 1:00

In [30]:
```
print(ON_vacuum_with_noise)
display(Audio(ON_vacuum_with_noise))
```

ON_vacuum_with_noise.wav

0:00 / 0:59

In [31]:
```
print(ON_air_leaking)
display(Audio(ON_air_leaking))
```

ON_air_leaking.wav

0:00 / 1:00

In [32]:
```
print(ON_air_leaking_with_noise)
display(Audio(ON_air_leaking_with_noise))
```

ON_air_leaking_with_noise.wav

0:00 / 0:59

Task 2.1

After listening to the sound according to each state (condition), do you think you can figure out the state based only on sound? Describe how you hear the sounds depending on the states.

State	Description
OFF	Low consistent frequency
OFF_with_noise	Low consistent frequency with a grainy quality at a higher frequency. Almost like driving on a road.
ON_vacuum	Higher consistent frequency and louder than the OFF state
ON_vacuum_with_noise	High consistent frequency with additional ringing tone.
ON_air_leaking	High frequency tone with a low frequency background.
ON_air_leaking_with_noise	High frequency tone with low frequency background and grainy quality.

Signal processing

You definitely tell the differences between OFF and ON states. In addition, you probably "feel" different sounds between ON/vacuuming and ON/air-leaking states. Using signal processing, let's analyze data to see how signals are different. To analyze audio, we will mainly use Python librosa package. The librosa is a Python package for music and audio analysis.

Figure 8 illustrates the procedure of development of a machine learning model in a typical and simple. As we did in Lab 8 and Lab 9, there are many ways to perform signal processing. Even we can use raw data for a feature for the training model input. This process, making input data for a machine learning model, is normally called feature extraction or feature engineering. When determining the feature, we have to consider many aspects, such as data flow, dimension, coherence, efficiency, effectiveness, representativeness, performance, and so on. As a practice in this lab, we will only use one feature, however, it is highly recommended to try other features for training the model. In addition, upon the

evaluation, we are able to optimize not only feature extraction method but also machine learning model (hyper parameters, architecture, and so on) for the best performance.

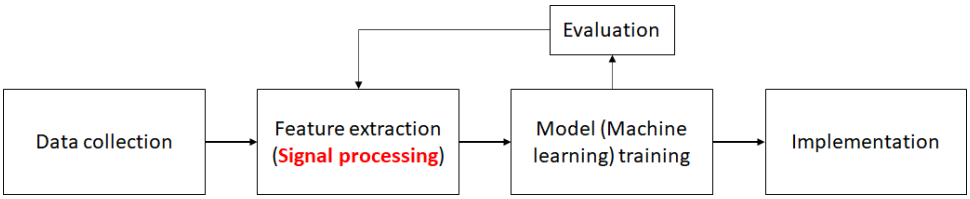


Figure 8 Development procedure of a typical and simple machine learning model

As a feature for the model, we will use Mel-spectrogram. The Mel-scale is a human perceived frequency of a pure tone to its actual measured frequency. Humans are much better to tell subtle changes at low frequencies than at high frequencies. Mel-spectrogram is more suitable for human auditory sensing that indicates the linear distirubtion under 1,000 Hz and the logarithm growth above 1,000 Hz, which result in many ASR (automatic speech recognition) techniques adopt the Mel-scale as a step of feature extractions.

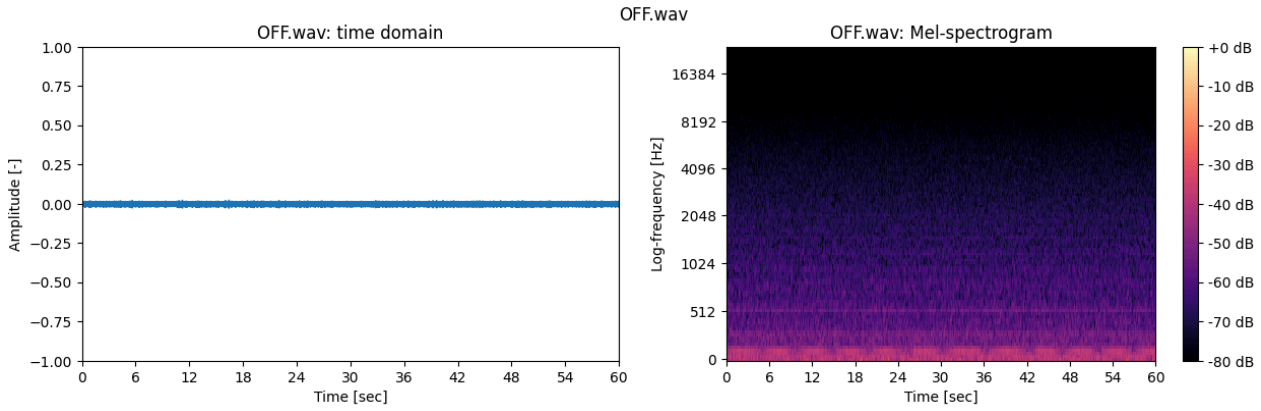
In the following code blocks, let's visualize data and practice Mel-spectrogram process. Of course, you can create your own method for the signal processing using Python basic functions, NumPy, SciPy, and so on. However, we will take advantage of the librosa package. Visit librosa.feature.melspectrogram and Understanding the Mel Spectrogram to figure out the method and Mel-spectrogram.

In [33]:
```
import librosa
import librosa.display
import matplotlib.pyplot as plt
import numpy as np

N_FFT = 2048  # Number of FFT points, because MTConnect stream chunk size is 2048. This is a constant.
y_off, sr = librosa.load(
    OFF, sr=48000
)  # y_off is audio data in off state and sr is sampling rate
M_off = librosa.feature.melspectrogram(
    y=y_off,
    sr=sr,
    n_fft=N_FFT,
    hop_length=int(N_FFT / 4),
    win_length=N_FFT,
    window="hann",
    n_mels=128,
)  # This is Mel-spectrogram
```

In [34]:
```
# This example plots time-domain (raw signal) on the left-hand side and spectrogram on the right-hand side
# Note that legend (color) means relative signal intensity in dB unit in the spectrogram.
# Also, in the spectrogram, the y-axis (frequency) is log scale.
fig, ax = plt.subplots(1, 2, sharex=True, figsize=(15, 4))
librosa.display.waveshow(y_off, sr=sr, ax=ax[0])  # time domain (raw data) plot
img = librosa.display.specshow(
    librosa.power_to_db(2 * abs(M_off) / N_FFT, ref=1),
    ax=ax[1],
    x_axis="time",
    y_axis="mel",
    sr=sr,
    vmin=-80,
    vmax=0,
)  # mel-spectrogram plot
ax[0].set(
    title=OFF + ": time domain",
    ylim=[-1, 1],
    xlim=[0, 60],
    ylabel="Amplitude [-]",
    xlabel="Time [sec]",
)
ax[1].set(
    title=OFF + ": Mel-spectrogram",
```

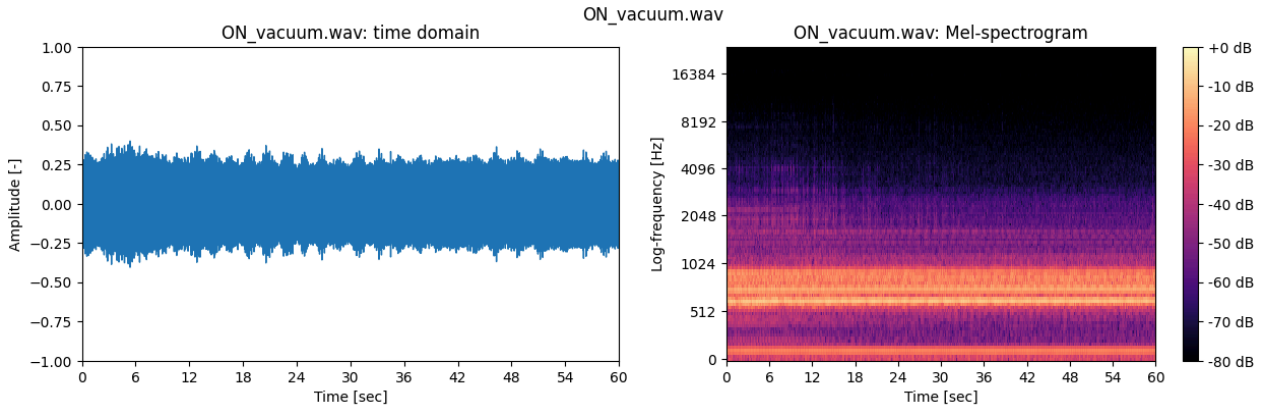
```
    ylabel="Log-frequency [Hz]",
    xlabel="Time [sec]",
)
cbar = fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(OFF)
```

Out[34]: Text(0.5, 0.98, 'OFF.wav')



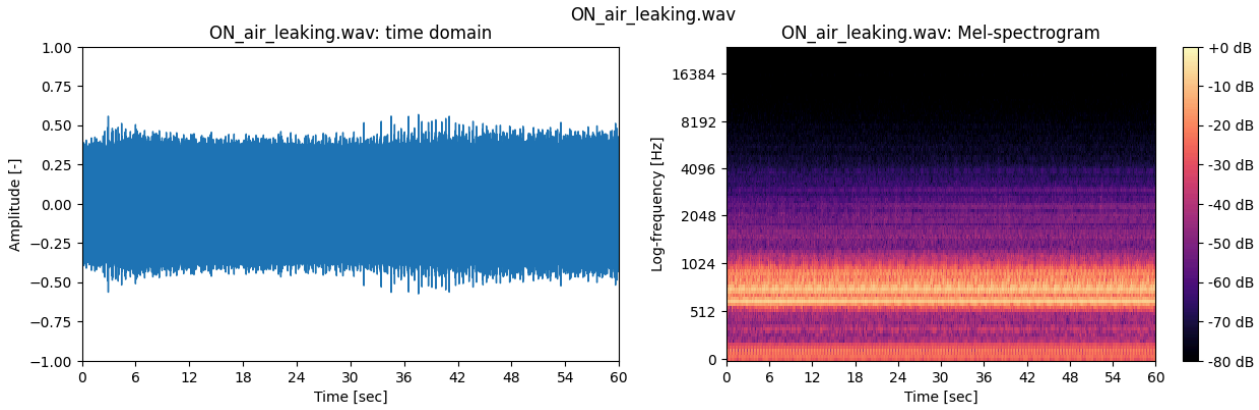
In [35]:
```
# This example is for plotting ON/vaccum

y_vacuum, sr = librosa.load(
    ON_vacuum, sr=48000
)  # y_vacuum is audio data in on/vacuuming state and sr is sampling rate
M_vacuum = librosa.feature.melspectrogram(
    y=y_vacuum,
    sr=sr,
    n_fft=N_FFT,
    hop_length=int(N_FFT / 4),
    win_length=N_FFT,
    window="hann",
    n_mels=128,
)

fig, ax = plt.subplots(1, 2, sharex=True, figsize=(15, 4))
librosa.display.waveshow(
    y_vacuum, sr=sr, ax=ax[0]
)  # time domain (raw data) plot
img = librosa.display.specshow(
    librosa.power_to_db(2 * abs(M_vacuum) / N_FFT, ref=1),
    ax=ax[1],
    x_axis="time",
    y_axis="mel",
    sr=sr,
    vmin=-80,
    vmax=0,
)  # mel-spectrogram plot
ax[0].set(
    title=ON_vacuum + ": time domain",
    ylim=[-1, 1],
    xlim=[0, 60],
    ylabel="Amplitude [-]",
    xlabel="Time [sec]",
)
ax[1].set(
    title=ON_vacuum + ": Mel-spectrogram",
    ylabel="Log-frequency [Hz]",
    xlabel="Time [sec]",
)
cbar = fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(ON_vacuum)
```

Out[35]: Text(0.5, 0.98, 'ON_vacuum.wav')



In [36]:
```
# This example is for plotting ON/leaking

y_airleaking, sr = librosa.load(
    ON_air_leaking, sr=48000
)  # y_airleaking is audio data in on/leaking state and sr is sampling rate
M_airleaking = librosa.feature.melspectrogram(
    y=y_airleaking,
    sr=sr,
    n_fft=N_FFT,
    hop_length=int(N_FFT / 4),
    win_length=N_FFT,
    window="hann",
    n_mels=128,
)

fig, ax = plt.subplots(1, 2, sharex=True, figsize=(15, 4))
librosa.display.waveshow(
    y_airleaking, sr=sr, ax=ax[0]
)  # time domain (raw data) plot
img = librosa.display.specshow(
    librosa.power_to_db(2 * abs(M_airleaking) / N_FFT, ref=1),
    ax=ax[1],
    x_axis="time",
    y_axis="mel",
    sr=sr,
    vmin=-80,
    vmax=0,
)  # mel-spectrogram plot
ax[0].set(
    title=ON_air_leaking + ": time domain",
    ylim=[-1, 1],
    xlim=[0, 60],
    ylabel="Amplitude [-]",
    xlabel="Time [sec]",
)
ax[1].set(
    title=ON_air_leaking + ": Mel-spectrogram",
    ylabel="Log-frequency [Hz]",
    xlabel="Time [sec]",
)
cbar = fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(ON_air_leaking)
```

Out[36]: Text(0.5, 0.98, 'ON_air_leaking.wav')



Task 2.2

Using the code blocks above, plot other data below.

- OFF_with_noise.wav
- ON_vacuum_with_noise.wav
- ON_air_leaking_with_noise_wave.wav

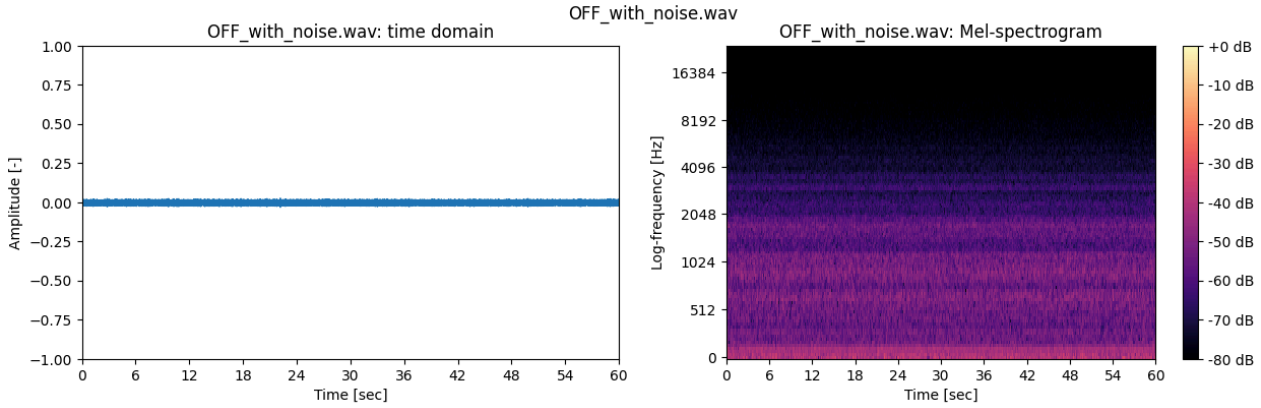
By comparing and analyzing data, discuss differences between the states.

In [37]:

```
## variables for the following session and this task

y_off_noise, sr = librosa.load(
    OFF_with_noise, sr=48000
)  # y_off is audio data in off state and sr is sampling rate
M_off_noise = librosa.feature.melspectrogram(
    y=y_off_noise,
    sr=sr,
    n_fft=N_FFT,
    hop_length=int(N_FFT / 4),
    win_length=N_FFT,
    window="hann",
    n_mels=128,
)  # This is Mel-spectrogram

fig, ax = plt.subplots(1, 2, sharex=True, figsize=(15, 4))
librosa.display.waveshow(
    y_off_noise, sr=sr, ax=ax[0]
)  # time domain (raw data) plot
img = librosa.display.specshow(
    librosa.power_to_db(2 * abs(M_off_noise) / N_FFT, ref=1),
    ax=ax[1],
    x_axis="time",
    y_axis="mel",
    sr=sr,
    vmin=-80,
    vmax=0,
)  # mel-spectrogram plot
ax[0].set(
    title=OFF_with_noise + ": time domain",
    ylim=[-1, 1],
    xlim=[0, 60],
    ylabel="Amplitude [-]",
    xlabel="Time [sec]",
)
ax[1].set(
    title=OFF_with_noise + ": Mel-spectrogram",
    ylabel="Log-frequency [Hz]",
    xlabel="Time [sec]",
)
```

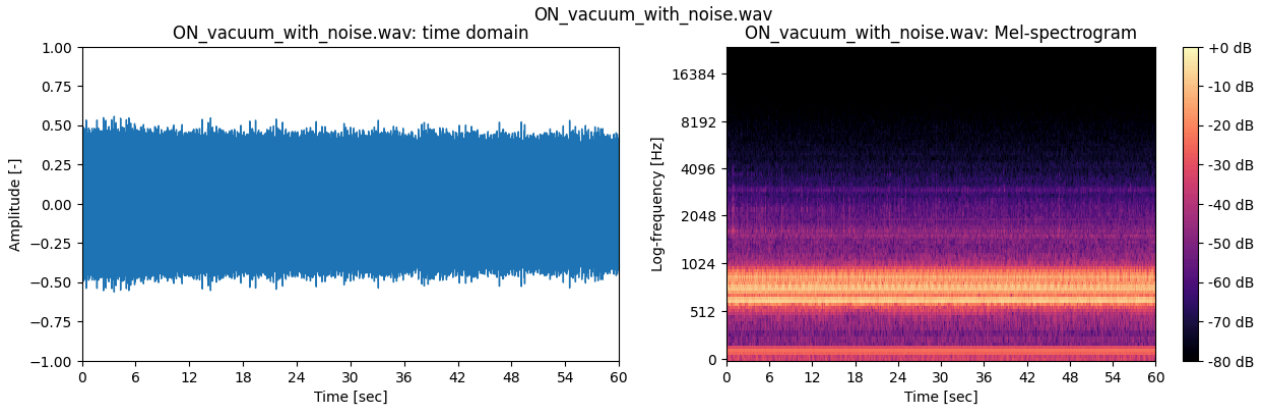
```
cbar = fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(OFF_with_noise)
```

Out[37]: Text(0.5, 0.98, 'OFF_with_noise.wav')



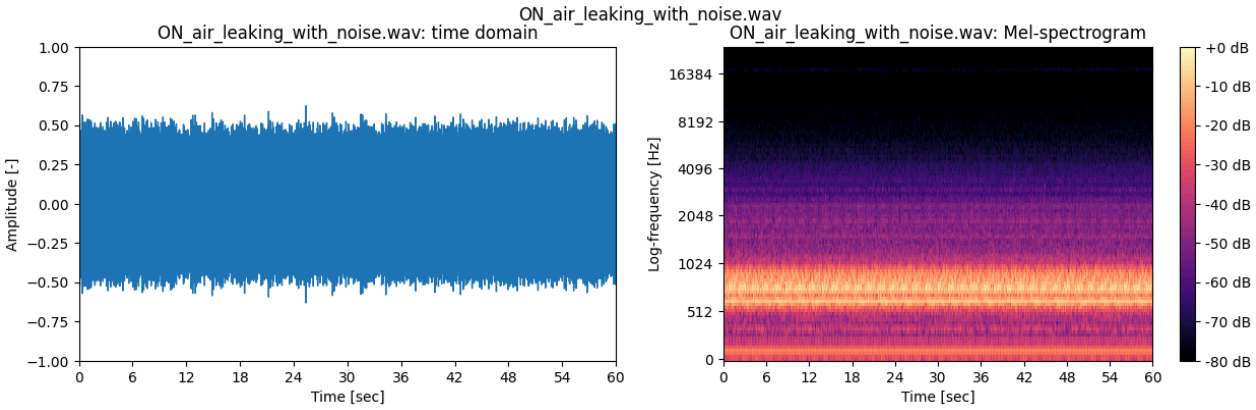
In [38]:
```
## Add more code blocks for Task 2.2 if needed
y_vacuum_noise, sr = librosa.load(
    ON_vacuum_with_noise, sr=48000
)  # y_vacuum is audio data in on/vacuuming state and sr is sampling rate
M_vacuum_noise = librosa.feature.melspectrogram(
    y=y_vacuum_noise,
    sr=sr,
    n_fft=N_FFT,
    hop_length=int(N_FFT / 4),
    win_length=N_FFT,
    window="hann",
    n_mels=128,
)

fig, ax = plt.subplots(1, 2, sharex=True, figsize=(15, 4))
librosa.display.waveshow(
    y_vacuum_noise, sr=sr, ax=ax[0]
)  # time domain (raw data) plot
img = librosa.display.specshow(
    librosa.power_to_db(2 * abs(M_vacuum_noise) / N_FFT, ref=1),
    ax=ax[1],
    x_axis="time",
    y_axis="mel",
    sr=sr,
    vmin=-80,
    vmax=0,
)  # mel-spectrogram plot
ax[0].set(
    title=ON_vacuum_with_noise + ": time domain",
    ylim=[-1, 1],
    xlim=[0, 60],
    ylabel="Amplitude [-]",
    xlabel="Time [sec]",
)
ax[1].set(
    title=ON_vacuum_with_noise + ": Mel-spectrogram",
    ylabel="Log-frequency [Hz]",
    xlabel="Time [sec]",
)
cbar = fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(ON_vacuum_with_noise)
```

Out[38]: Text(0.5, 0.98, 'ON_vacuum_with_noise.wav')



In [39]:
```
y_airleaking_noise, sr = librosa.load(
    ON_air_leaking_with_noise, sr=48000
)  # y_vacuum is audio data in on/vacuuming state and sr is sampling rate
M_airleaking_noise = librosa.feature.melspectrogram(
    y=y_airleaking_noise,
    sr=sr,
    n_fft=N_FFT,
    hop_length=int(N_FFT / 4),
    win_length=N_FFT,
    window="hann",
    n_mels=128,
)

fig, ax = plt.subplots(1, 2, sharex=True, figsize=(15, 4))
librosa.display.waveshow(
    y_airleaking_noise, sr=sr, ax=ax[0]
)  # time domain (raw data) plot
img = librosa.display.specshow(
    librosa.power_to_db(2 * abs(M_airleaking_noise) / N_FFT, ref=1),
    ax=ax[1],
    x_axis="time",
    y_axis="mel",
    sr=sr,
    vmin=-80,
    vmax=0,
)  # mel-spectrogram plot
ax[0].set(
    title=ON_air_leaking_with_noise + ": time domain",
    ylim=[-1, 1],
    xlim=[0, 60],
    ylabel="Amplitude [-]",
    xlabel="Time [sec]",
)
ax[1].set(
    title=ON_air_leaking_with_noise + ": Mel-spectrogram",
    ylabel="Log-frequency [Hz]",
    xlabel="Time [sec]",
)
cbar = fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(ON_air_leaking_with_noise)
```

Out[39]: Text(0.5, 0.98, 'ON_air_leaking_with_noise.wav')



Both off states are low frequency on the Log-Frequency scale with a small range of amplitude. They are both quiet with the off state with noise being a bit louder. Comparing the vacuum and air leaking states, they both have louder bands between 512 and 1024 Hz as well as a low band close to 0 Hz. However, the time domain graph shows that the vacuum state has a more regular pattern than the air leaking state however when noise is added to both states it becomes harder to distinguish them from one another.

Labeling and CNN model data input pipeline

Let's make the input data with labeling (output) for the CNN model training. Before discretizing data for the inputs, let's understand how we split data. First we set the size of window (window size), so-called input data length. And then scan the entire dataset with a scanning interval as shown in Figure 9.

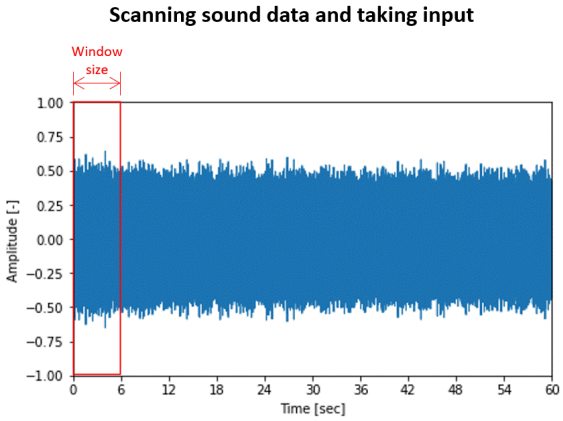


Figure 9 Scanning for taking input data

As an example, let's have the input data 1 second. However, when it comes to the MTConnect straem, the chunk size (minimum data length in a sequence) is 2048 data points, which means the chunk is 0.042667 sec because the sampling rate is 48000 Hz. We calculate how many chunks (N_{seq}) are close to 1 second. The number of chunks close to 1 second is 23 (actually 0.98133 sec). The number of data points for the window size is 47,104 which can be calculated by multiplying the chunk size and the number of the sequences ($N_{chunk} \times N_{seq}$). Also, let's have a scanning interval to equal the chunk size. That means in every interval size, 1-second data is taken as input data after performing signal processing. The signal processing (feature extraction) is the same as in the section above, Mel-spectrogram. Input data specifications and feature extraction are summarized below.

- Window size: 0.981333 sec
  - N_{chunk} = 2048

- N_{seq} = 23
- f_s (sampling frequency) = 48000 Hz
- t_{window} = 0.98133 sec. ($N_{chunk} \times N_{seq}/f_s$)
- Scanning interval = 0.042667 sec (= 2048 data points)

- Feature: Mel-spectrogram
  - N_FFT (number of FFT points) = 2048
  - hop_length = 512
  - Windowing function = hanning
  - n_mels (number of Mel-filter banks) = 128

It is expected that the input data has a 2D-array and the shape (dimension) is (128×93). With this information, let's perform splitting data by scanning the dataset for generating the machine learning input and labeling.

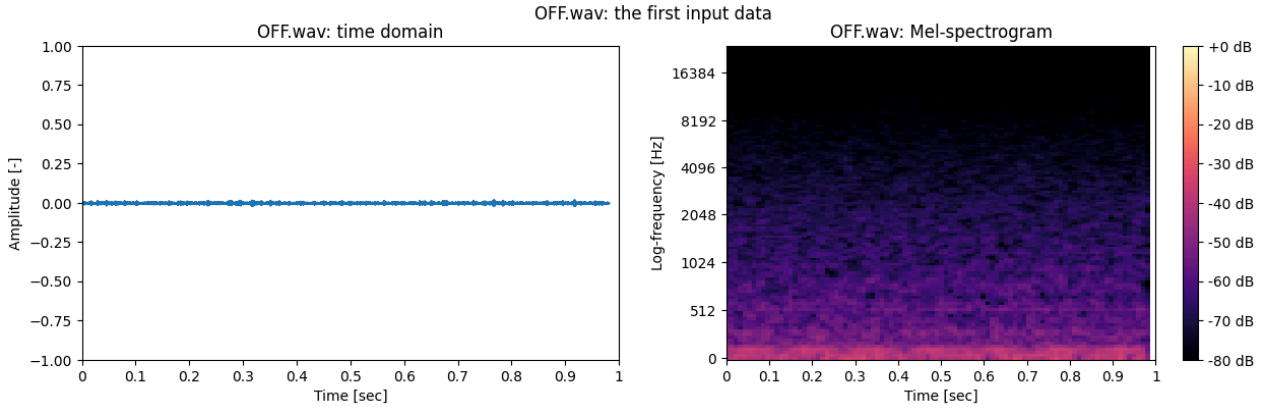
In [40]:
```
# This code prints out length of the audio file (OFF.WAV) in second unit.
print("Total data points of an audio file: ", len(y_off))
print("Total length of an audio file: ", len(y_off) / sr, "sec")

n_chunk = 2048  # Number of data points in a chunk
n_seq = 23  # Number of the required sequences for a window
n_interval = 2048  # Number of data points for interval
n_total_lengh = len(y_off)  # Number of total data points
n_window_length = n_chunk * n_seq  # number of data points in a window
N_window = (
    int((len(y_off) - n_window_length) / n_interval) + 1
)  # Number of windows from the data

print("\nTotal number of windows for training data input per file:", N_window)
print("Length of each window:", n_window_length / sr, "sec")
```

```
Total data points of an audio file:  2880000
Total length of an audio file:  60.0 sec

Total number of windows for training data input per file: 1384
Length of each window: 0.9813333333333333 sec
```

In [41]:
```
# Let's conduct feature extraction and visualization for the first data of OFF.wav
y_off_1 = y_off[0 : n_chunk * n_seq]  # taking the first window by indexing
M_off_1 = librosa.feature.melspectrogram(
    y=y_off_1,
    sr=sr,
    n_fft=N_FFT,
    hop_length=int(N_FFT / 4),
    win_length=N_FFT,
    window="hann",
    n_mels=128,
)
S_off_1 = 2 * abs(M_off_1) / N_FFT  # power of Mel-spectrogram as input

fig, ax = plt.subplots(1, 2, sharex=True, figsize=(15, 4))
librosa.display.waveshow(
    y_off_1, sr=sr, ax=ax[0]
)  # time domain (raw data) plot
img = librosa.display.specshow(
    librosa.power_to_db(S_off_1, ref=1),
    ax=ax[1],
    x_axis="time",
    y_axis="mel",
    sr=sr,
    vmin=-80,
    vmax=0,
)  # mel-spectrogram plot in dB unit
ax[0].set(
    title=OFF + ": time domain",
    ylim=[-1, 1],
    xlim=[0, 1],
    ylabel="Amplitude [-]",
```

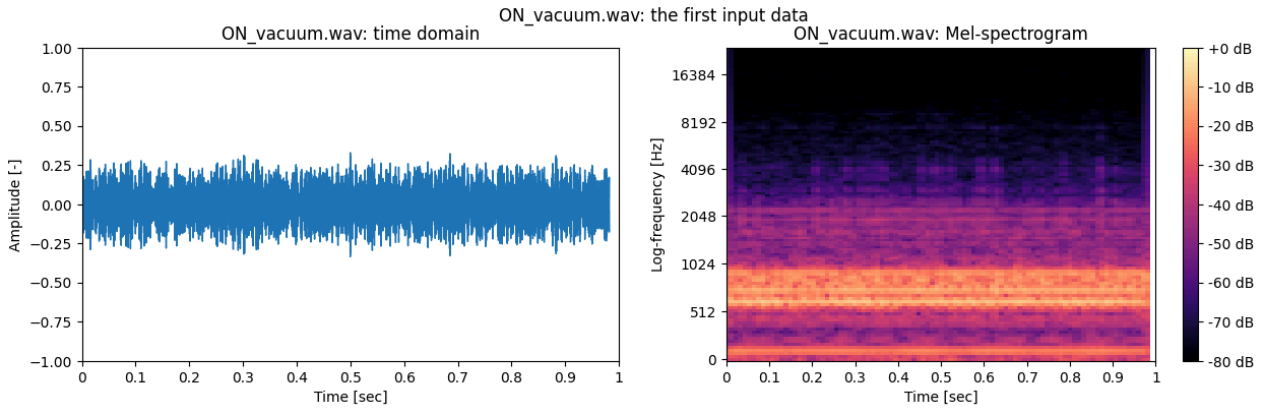
```
    xlabel="Time [sec]",
)
ax[1].set(
    title=OFF + ": Mel-spectrogram",
    ylabel="Log-frequency [Hz]",
    xlabel="Time [sec]",
)
cbar = fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(OFF + ": the first input data")
```

Out[41]: Text(0.5, 0.98, 'OFF.wav: the first input data')



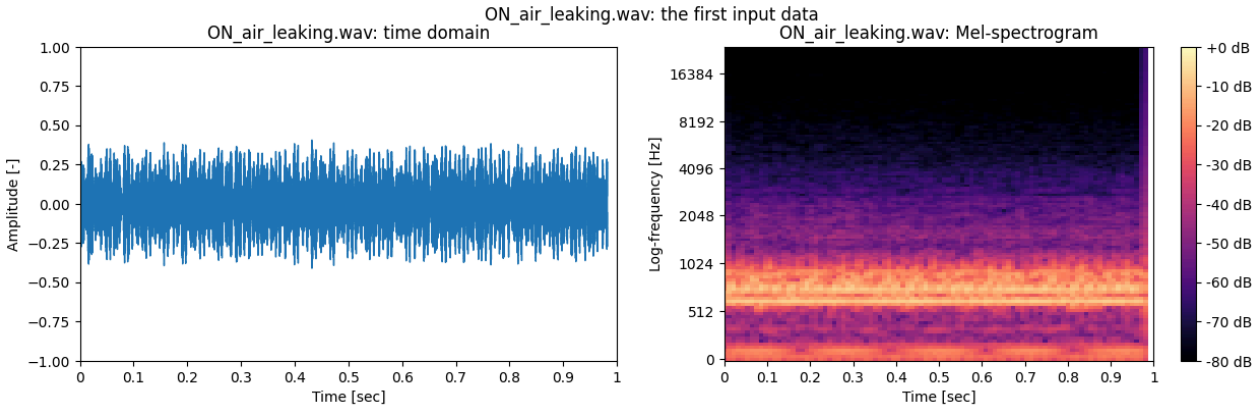
In [42]:
```
# Let's conduct feature extraction and visualization for the first data of ON_vacuuming.wav
y_vacuum_1 = y_vacuum[
    0 : n_chunk * n_seq
]  # taking the first window by indexing
M_vacuum_1 = librosa.feature.melspectrogram(
    y=y_vacuum_1,
    sr=sr,
    n_fft=N_FFT,
    hop_length=int(N_FFT / 4),
    win_length=N_FFT,
    window="hann",
    n_mels=128,
)
S_vacuum_1 = 2 * abs(M_vacuum_1) / N_FFT  # power of Mel-spectrogram as input

fig, ax = plt.subplots(1, 2, sharex=True, figsize=(15, 4))
librosa.display.waveshow(
    y_vacuum_1, sr=sr, ax=ax[0]
)  # time domain (raw data) plot
img = librosa.display.specshow(
    librosa.power_to_db(S_vacuum_1, ref=1),
    ax=ax[1],
    x_axis="time",
    y_axis="mel",
    sr=sr,
    vmin=-80,
    vmax=0,
)  # mel-spectrogram plot in dB unit
ax[0].set(
    title=ON_vacuum + ": time domain",
    ylim=[-1, 1],
    xlim=[0, 1],
    ylabel="Amplitude [-]",
    xlabel="Time [sec]",
)
ax[1].set(
    title=ON_vacuum + ": Mel-spectrogram",
    ylabel="Log-frequency [Hz]",
    xlabel="Time [sec]",
)
cbar = fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(ON_vacuum + ": the first input data")
```

Out[42]: Text(0.5, 0.98, 'ON_vacuum.wav: the first input data')



In [43]:
```python
# Let's conduct feature extraction and visualization for the first data of ON_air_leaking.wav
y_airleaking_1 = y_airleaking[
    0 : n_chunk * n_seq
]  # taking the first window by indexing
M_airleaking_1 = librosa.feature.melspectrogram(
    y=y_airleaking_1,
    sr=sr,
    n_fft=N_FFT,
    hop_length=int(N_FFT / 4),
    win_length=N_FFT,
    window="hann",
    n_mels=128,
)
S_airleaking_1 = (
    2 * abs(M_airleaking_1) / N_FFT
)  # power of Mel-spectrogram as input

fig, ax = plt.subplots(1, 2, sharex=True, figsize=(15, 4))
librosa.display.waveshow(
    y_airleaking_1, sr=sr, ax=ax[0]
)  # time domain (raw data) plot
img = librosa.display.specshow(
    librosa.power_to_db(S_airleaking_1, ref=1),
    ax=ax[1],
    x_axis="time",
    y_axis="mel",
    sr=sr,
    vmin=-80,
    vmax=0,
)  # mel-spectrogram plot in dB unit
ax[0].set(
    title=ON_air_leaking + ": time domain",
    ylim=[-1, 1],
    xlim=[0, 1],
    ylabel="Amplitude [-]",
    xlabel="Time [sec]",
)
ax[1].set(
    title=ON_air_leaking + ": Mel-spectrogram",
    ylabel="Log-frequency [Hz]",
    xlabel="Time [sec]",
)
cbar = fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(ON_air_leaking + ": the first input data")
```

Out[43]: Text(0.5, 0.98, 'ON_air_leaking.wav: the first input data')



Task 2.3

The first input data processing from an audio file is given above, can you take the second, and third input data? (*Hint: use indexing of the raw data (y)*)

How to generalize/express the process for n^{th} data input?

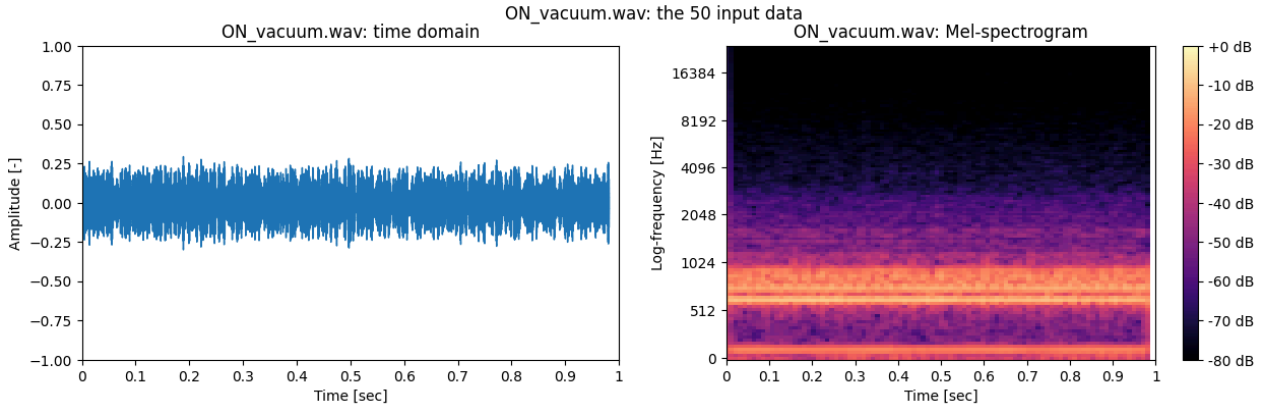
To generalize or express the process for n^{th} data input, you can initialize an index or n variable to represent the value of n and then index the data from n until n + 1, multiplied by the length of a window. In the code below I am using a zero-index but this can be adjusted per user preference.

In [84]:
```
### You can use this code block to test Task 2.3
### Try it!

idx = 50  # index of the window for visualization
```

In [85]:
```
y_vacuum_1 = y_vacuum[
    n_window_length * idx : n_window_length * (idx + 1)
]  # taking the first window by indexing
M_vacuum_1 = librosa.feature.melspectrogram(
    y=y_vacuum_1,
    sr=sr,
    n_fft=N_FFT,
    hop_length=int(N_FFT / 4),
    win_length=N_FFT,
    window="hann",
    n_mels=128,
)
S_vacuum_1 = 2 * abs(M_vacuum_1) / N_FFT  # power of Mel-spectrogram as input

fig, ax = plt.subplots(1, 2, sharex=True, figsize=(15, 4))
librosa.display.waveshow(
    y_vacuum_1, sr=sr, ax=ax[0]
)  # time domain (raw data) plot
img = librosa.display.specshow(
    librosa.power_to_db(S_vacuum_1, ref=1),
    ax=ax[1],
    x_axis="time",
    y_axis="mel",
    sr=sr,
    vmin=-80,
    vmax=0,
)  # mel-spectrogram plot in dB unit
ax[0].set(
    title=ON_vacuum + ": time domain",
    ylim=[-1, 1],
    xlim=[0, 1],
    ylabel="Amplitude [-]",
```

```
    xlabel="Time [sec]",
)
ax[1].set(
    title=ON_vacuum + ": Mel-spectrogram",
    ylabel="Log-frequency [Hz]",
    xlabel="Time [sec]",
)
cbar = fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(ON_vacuum + f": the {idx} input data")
```

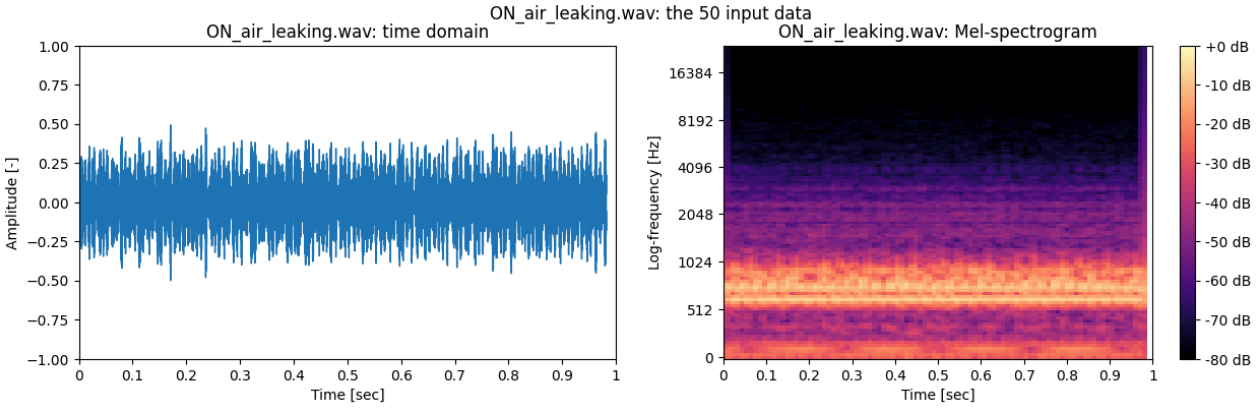
Out[85]: Text(0.5, 0.98, 'ON_vacuum.wav: the 50 input data')



In [86]:
```
y_airleaking_1 = y_airleaking[
    n_window_length * idx : n_window_length * (idx + 1)
]  # taking the first window by indexing
M_airleaking_1 = librosa.feature.melspectrogram(
    y=y_airleaking_1,
    sr=sr,
    n_fft=N_FFT,
    hop_length=int(N_FFT / 4),
    win_length=N_FFT,
    window="hann",
    n_mels=128,
)
S_airleaking_1 = (
    2 * abs(M_airleaking_1) / N_FFT
)  # power of Mel-spectrogram as input

fig, ax = plt.subplots(1, 2, sharex=True, figsize=(15, 4))
librosa.display.waveshow(
    y_airleaking_1, sr=sr, ax=ax[0]
)  # time domain (raw data) plot
img = librosa.display.specshow(
    librosa.power_to_db(S_airleaking_1, ref=1),
    ax=ax[1],
    x_axis="time",
    y_axis="mel",
    sr=sr,
    vmin=-80,
    vmax=0,
)  # mel-spectrogram plot in dB unit
ax[0].set(
    title=ON_air_leaking + ": time domain",
    ylim=[-1, 1],
    xlim=[0, 1],
    ylabel="Amplitude [-]",
    xlabel="Time [sec]",
)
ax[1].set(
    title=ON_air_leaking + ": Mel-spectrogram",
    ylabel="Log-frequency [Hz]",
    xlabel="Time [sec]",
)
```

```
cbar = fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(ON_air_leaking + f": the {idx} input data")
```

Out[86]: Text(0.5, 0.98, 'ON_air_leaking.wav: the 50 input data')



Now, let's build up the input pipeline for training model. The given function below, *get_input_data*, is to process feature extraction for making the training dataset.

In [45]:
```
def get_input_data(
    y, label, N_window, N_interval, N_seq, N_chunk, n_fft, hop_length, sr
):
    ## Args decription below
    # y is a raw data (sound signal) for being processed.
    # label is the state (condition), string dtype
    # N_window is number of windows to be splitted for input data from the raw data.
    # N_interval is the scanning interval.
    # N_seq is number of the seuqneces in MTConnect stream to have a specific length of window.
    # N_chunk is a chunk size in which a sequence of MTConnect has data points.
    # n_fft is number of FFT points.
    # hop_length is hop length when spectrogram calculation
    # sr is sampling rate of the audio.

    X = []  # initialize input data array
    Y = []  # initialize label data array
    for i in range(N_window):
        y_temp = y[i * N_interval : (i * N_interval) + N_chunk * N_seq]
        x_temp = librosa.feature.melspectrogram(
            y=y_temp,
            sr=sr,
            n_fft=n_fft,
            hop_length=hop_length,
            win_length=n_fft,
            window="hann",
            n_mels=128,
        )
        feature_temp = 2 * abs(x_temp) / n_fft
        Y.append(label)
        X.append(feature_temp)

    return np.array(X), np.array(Y)
```

In [46]:
```
# X_off and Y_off are training input data and labels, respectively.
# It takes tens of seconds. If you create your own function to split data by taking not raw data but spectro
X_off, Y_off = get_input_data(
    y=y_off,
    label="OFF",
    N_window=N_window,
    N_interval=n_interval,
    N_seq=n_seq,
    N_chunk=n_chunk,
    n_fft=N_FFT,
    hop_length=int(N_FFT / 4),
```

```
    sr=sr,
)

print("X_off shape is", X_off.shape)
print("Y_off shape is", Y_off.shape)
```

```
X_off shape is (1384, 128, 93)
Y_off shape is (1384,)
```

Let's understand the input data after processing (Figure 10). X_off (training input) has (k, n, m) shape, (1384, 128, 93) in this case. Y_off (training label) has (1384,) shape. This means the number of data for training for "OFF" label is 1384. Each input data (window) has 128×93 size of 2D-array, which is the power of the Mel-spectrogram. The column, n=128, represents frequency (Mel) components and the row, m=93, represents time components. Because raw sound data itself in WAV format is normalized from -1 to 1, we don't need to normalize them for training input.

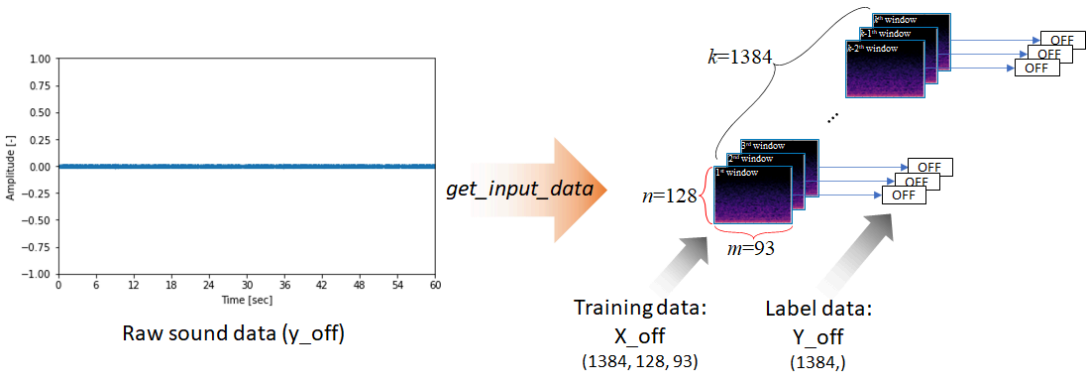


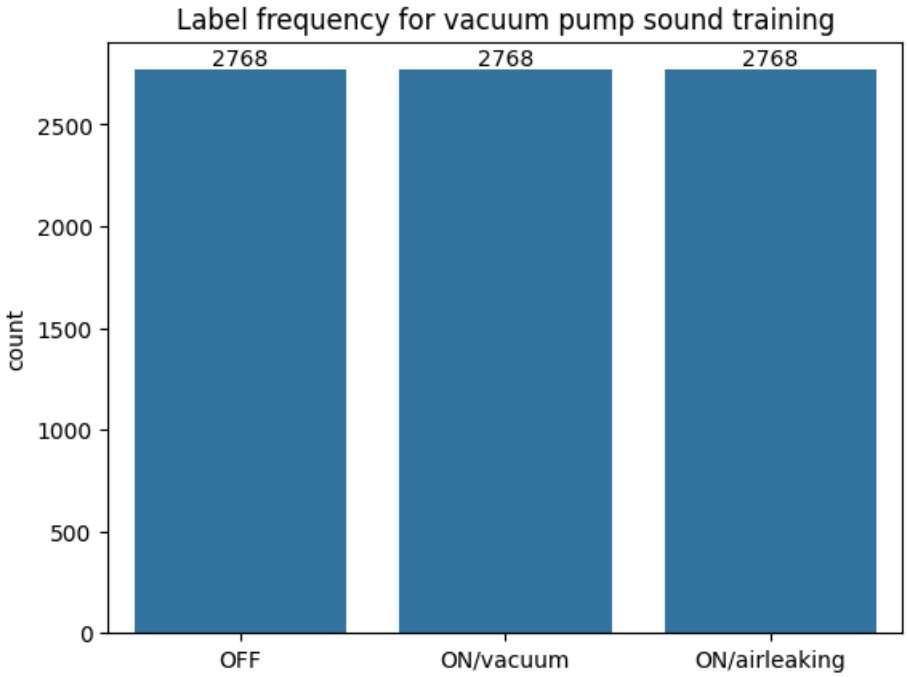
Figure 10 Understanding processing and shape of input data for training

In [47]:
```
### keep working on other data, this may take a few minutes

X_off_noise, Y_off_noise = get_input_data(
    y=y_off_noise,
    label="OFF",
    N_window=N_window,
    N_interval=n_interval,
    N_seq=n_seq,
    N_chunk=n_chunk,
    n_fft=N_FFT,
    hop_length=int(N_FFT / 4),
    sr=sr,
)

X_vacuum, Y_vacuum = get_input_data(
    y=y_vacuum,
    label="ON/vacuum",
    N_window=N_window,
    N_interval=n_interval,
    N_seq=n_seq,
    N_chunk=n_chunk,
    n_fft=N_FFT,
    hop_length=int(N_FFT / 4),
    sr=sr,
)
X_vacuum_noise, Y_vacuum_noise = get_input_data(
    y=y_vacuum_noise,
    label="ON/vacuum",
    N_window=N_window,
    N_interval=n_interval,
    N_seq=n_seq,
    N_chunk=n_chunk,
    n_fft=N_FFT,
    hop_length=int(N_FFT / 4),
    sr=sr,
)
```

```
X_airleaking, Y_airleaking = get_input_data(
    y=y_airleaking,
    label="ON/airleaking",
    N_window=N_window,
    N_interval=n_interval,
    N_seq=n_seq,
    N_chunk=n_chunk,
    n_fft=N_FFT,
    hop_length=int(N_FFT / 4),
    sr=sr,
)
X_airleaking_noise, Y_airleaking_noise = get_input_data(
    y=y_airleaking_noise,
    label="ON/airleaking",
    N_window=N_window,
    N_interval=n_interval,
    N_seq=n_seq,
    N_chunk=n_chunk,
    n_fft=N_FFT,
    hop_length=int(N_FFT / 4),
    sr=sr,
)
```

In [48]:
```
# concatenate all data in training data
# number of each input dataset is 1348, and we have total 6 data which has same size
# therefore, number of training data will be 8088
X_data = np.concatenate(
    (
        X_off,
        X_off_noise,
        X_vacuum,
        X_vacuum_noise,
        X_airleaking,
        X_airleaking_noise,
    ),
    axis=0,
)
Y_data = np.concatenate(
    (
        Y_off,
        Y_off_noise,
        Y_vacuum,
        Y_vacuum_noise,
        Y_airleaking,
        Y_airleaking_noise,
    ),
    axis=0,
)
```

In [49]:
```
import seaborn as sns

# seaborn is a statictical data visualization Python package

# before we plot the label frequency, we can expect how much data is in each label.
# Calculate the frequencies before you run this code.
# Are the results as you expected?

plt.figure()
plt.title("Label frequency for vacuum pump sound training")
ax = sns.countplot(x=Y_data)
for container in ax.containers:
    ax.bar_label(container)
plt.show()
```



To handling and evaluate training data, we will use scikit-learn Python package which is known as sklearn. Practically, sklearn, keras with TensorFlow are most widely used tool for ML training and implementation.

Before we split train and test (validation) dataset, let's encode the label data for training format. As defined, our problem (purpose) is a multi-class classification. Each class (normally formatted as positive integer) represents a state (operational condition) in this example.

In [50]:
```
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.utils import to_categorical

enc = LabelEncoder()  # encoder object
enc.fit(Y_data)
Y_data_enc = enc.transform(
    Y_data
)  # Encoding from string labels to integers; 0: OFF, 1: ON/airleaking, 2: ON/vacuum because encoding sort a
Y_label = to_categorical(
    Y_data_enc
)  # categorical labeling of the integer index; [1 0 0]: OFF, [0 1 0]: ON/airleaking, [0 0 1]: ON/vacuum
```

In [51]:
```
# using this code block,
# check the encoding performed as you expected.
# Note that, because order of encoding is sorted by ascending order,
# 0 represents "OFF", 1 represents "ON/airleaking", 2 represents "ON/vacuum"
# The encoded integer index means principle axis of the categorical matrix,
# for the model training, 0, 1, and 2 turn out [1 0 0], [0 1 0], and [0 0 1], respectively.

print("Y_data:", Y_data)
print("Encoded Y_data:", Y_data_enc)
print("Encoded classes:", enc.classes_)
print("Categorical transform:", Y_label)
```

```
Y_data: ['OFF' 'OFF' 'OFF' ... 'ON/airleaking' 'ON/airleaking' 'ON/airleaking']
Encoded Y_data: [0 0 0 ... 1 1 1]
Encoded classes: ['OFF' 'ON/airleaking' 'ON/vacuum']
Categorical transform: [[1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 ...
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]]
```

In [52]:
```
# Let's split training and testing dataset.
# X_train and Y_train will be used for training weights, bias, filters and so on.
# X_test and Y_test will be used for validating while training.
# In this example, we have 20% test data size. Therefore, the training data is 80%.

from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(
    X_data, Y_label, test_size=0.2, random_state=40
)
print("Number of data for training: {}".format(len(X_train)))
print("Number of data for validation: {}".format(len(X_test)))
```

```
Number of data for training: 6643
Number of data for validation: 1661
```

Now, we are ready to start training CNN model.

2.2 CNN model training

For the operational state prediction of the vacuum pump, we will use a convolutional neural network (CNN or ConvNet) architecture. While CNNs are mostly used for image classification/recognition, they also have been widely applied for sound recognition and classification tasks. CNNs are a type of multi-layer neural networks that typically contain convolution layers, subsampling layers, and fully connected (FC) layers. Even if the networks are complex because of a large amount of connectivity, the use of shared wieghts within layers helps in reducing the number of parameters that should be trained. Parameter sharing makes CNNs greatly reduce the number of unique model parameters and significantly increase the size of the network without requiring corresponding increase in training data.

A CNN can have a few or hundreds of layers that each learn to detect different features of an image. Filters are applied to each training image at different resolutions, and output of each convoluted image is used as the input of the next layer. Visit Towards Data Science, and Analytics Vidhya to grasp more about CNN.

The CNN architecture in this example is shown in Figure 11. Red-colored parameters in the architecture are adjustable hyperparameters. We will use two convolutional layers and two hidden layers. If you are interested, try common CNN archtectures, such as AlexNet, VGG, GoogleNet, and so on. As input, the CNN takes tensors of shape (freq. components, time components) of Mel-spectrogram. The output node will be one among "OFF" (0), "ON/airleaking" (1), and "ON/vacuum" (2). Let's create the CNN architecture using TensorFlow and Keras API.

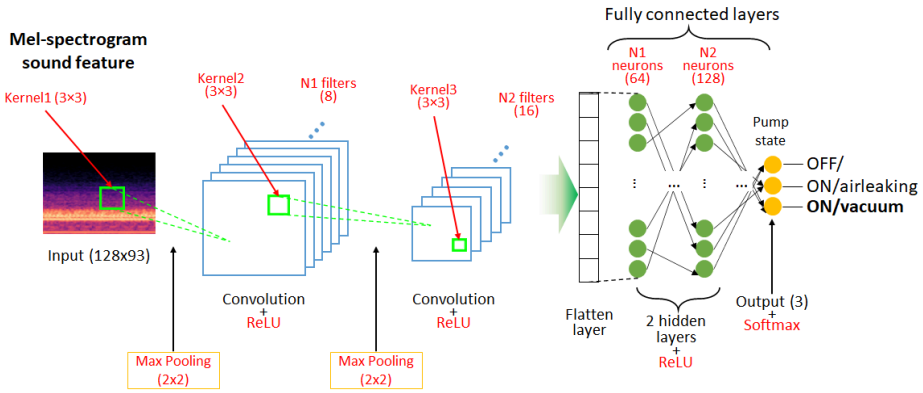


Figure 10 CNN architecture for vacuum pump operational state classification

Creating CNN model

In [53]:
```
import tensorflow as tf
```

In [54]:
```
from tensorflow.keras import layers

model = tf.keras.Sequential()  # model obj. takes sequential model
```

```
model.add(
    layers.Reshape(
        (X_train.shape[1], X_train.shape[2], 1),
        input_shape=(X_train.shape[1], X_train.shape[2]),
    )
)  # reshaping the input to ingest 2D CNN layer
model.add(
    layers.Conv2D(filters=8, kernel_size=(3, 3), activation="relu")
)  # first 2D CNN layer
model.add(layers.MaxPooling2D(pool_size=(3, 3)))  # max pooling layer
model.add(
    layers.Conv2D(filters=16, kernel_size=(3, 3), activation="relu")
)  # second 2D CNN layer
model.add(layers.MaxPooling2D(pool_size=(3, 3)))  # max pooling layer
model.add(layers.Flatten())  # flatten layer
model.add(layers.Dense(64, activation="relu"))  # first hidden layer
model.add(layers.Dense(128, activation="relu"))  # second hidden layer
model.add(
    layers.Dense(units=Y_train.shape[1], activation="softmax", name="output")
)  # output layer (node)
model.summary()  # This will show the model object
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/reshape.py:38: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
reshape (Reshape)	(None, 128, 93, 1)	0
conv2d (Conv2D)	(None, 126, 91, 8)	80
max_pooling2d (MaxPooling2D)	(None, 42, 30, 8)	0
conv2d_1 (Conv2D)	(None, 40, 28, 16)	1,168
max_pooling2d_1 (MaxPooling2D)	(None, 13, 9, 16)	0
flatten (Flatten)	(None, 1872)	0
dense (Dense)	(None, 64)	119,872
dense_1 (Dense)	(None, 128)	8,320
output (Dense)	(None, 3)	387

Total params: 129,827 (507.14 KB)

Trainable params: 129,827 (507.14 KB)

Non-trainable params: 0 (0.00 B)

Compiling and training the model

By running the code block below, let's compile the CNN model and then train it!

In [55]:
```
## Other hyperparameters
BATCH = 256  # the number of samples per gradient update
EPOCH = 5  # how many cycles will be trained in an entire dataset passed forward and backward through neural
LR = 10e-4  # learning rate is to control how much to change the model in response to the estimated error pe

# compiling the CNN model, loss function is categorical crossentropy and training metric is accuracy
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=LR),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)
```

```
# This will start model training and take a few minutes
history = model.fit(
    X_train,
    Y_train,
    validation_data=(X_test, Y_test),
    batch_size=BATCH,
    epochs=EPOCH,
    verbose=1,
)
```

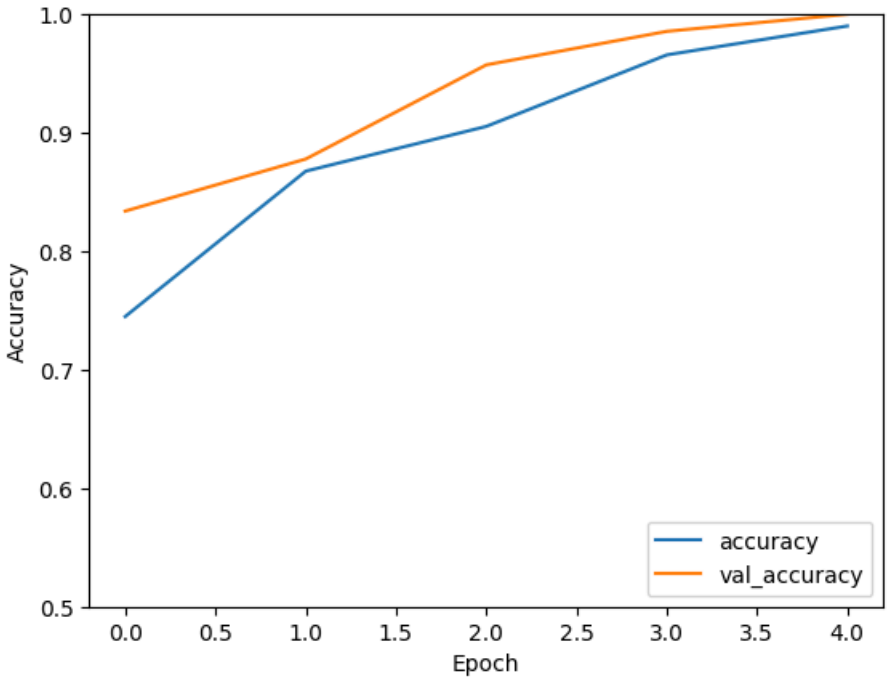
```
Epoch 1/5
26/26 ━━━━━━━━━━━━━━━━━━━━ 10s 187ms/step - accuracy: 0.7448 - loss: 0.9615 - val_accuracy: 0.8338 - val_los
s: 0.6074
Epoch 2/5
26/26 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - accuracy: 0.8675 - loss: 0.3442 - val_accuracy: 0.8778 - val_loss:
0.2523
Epoch 3/5
26/26 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step - accuracy: 0.9053 - loss: 0.2026 - val_accuracy: 0.9573 - val_loss:
0.1371
Epoch 4/5
26/26 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - accuracy: 0.9657 - loss: 0.1148 - val_accuracy: 0.9856 - val_loss:
0.0723
Epoch 5/5
26/26 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - accuracy: 0.9901 - loss: 0.0591 - val_accuracy: 1.0000 - val_loss:
0.0347
```

Evaluate model

Let's plot the training history and perform basic evaluation of the trained model on the test dataset.

In [56]:
```
# This is for accuracy in training history
plt.plot(history.history["accuracy"], label="accuracy")
plt.plot(history.history["val_accuracy"], label="val_accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.ylim([0.5, 1])
plt.legend(loc="lower right")
```

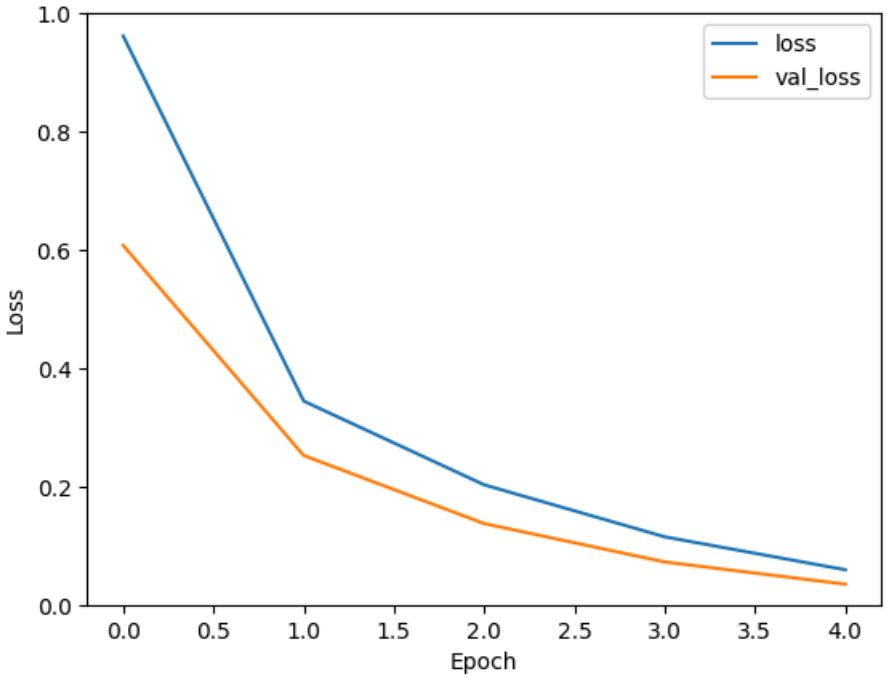
Out[56]:
```
<matplotlib.legend.Legend at 0x7f0cfe347ec0>
```



In [57]:
```
# This is for loss in training history
plt.plot(history.history["loss"], label="loss")
```

```
plt.plot(history.history["val_loss"], label="val_loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.ylim([0, 1])
plt.legend(loc="upper right")

test_loss, test_acc = model.evaluate(X_test, Y_test, verbose=2)
```

```
52/52 - 1s - 22ms/step - accuracy: 1.0000 - loss: 0.0347
```



Saving the trained model

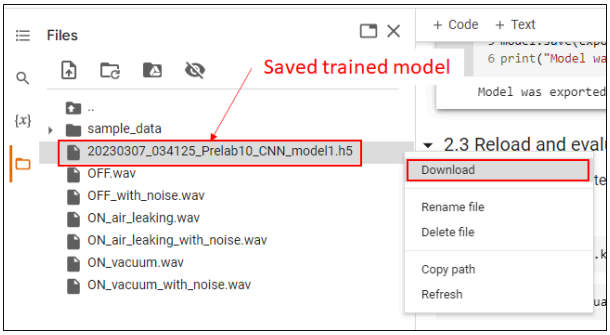
Then, let's save the trained model to implement it on Raspberry Pi. In this case we will use another standard binary format, Hierarchical Data Format so-called HDF5 or H5. Because Keras supports HDF5 format, we can take advantage of it.

In [58]:
```
import datetime

t = datetime.datetime.now().strftime(
    "%Y%m%d_%H%M%S"
)  # generate date time string to identify the model generated time
export_name = t + "_Prelab10_CNN_model1.h5"  # this is eventually filename.
model.save(export_name)  # save method save the model with the file name
print("Model was exported in this path: '{}'".format(export_name))
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)
`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.sav
e('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
Model was exported in this path: '20260409_004844_Prelab10_CNN_model1.h5'
```

2.3 Reload and evaluate the CNN model

Let's reload and evaluate the saved model. Before that, **download the model to your laptop from the Colab Files session as the capture below to use in the lab activity.**



Before we move on to reload and evaluate the model, let's figure out the evaluation means. For the machine learning model evaluation, we test it using various methods and performance metrics. We will check the confusion matrix, and some performance metrics (accuracy, precision, recall, and F1 score). Go through this article and perform Task 2.4.

Task 2.4

1. What is confusion matrix? Why is it used for evaluation?
2. Explain the performance metrics. * Accuracy * Precision * Recall * F1 score
3. Which performance metric above is most important for this problem and why?

A confusion matrix is a matrix representation of the proportion of times a model predicts a certain outcome compared to the actual outcome. For this example, it can be thought of as the proportion of times the model predicts a specific state compared to the actual state of the data. This matrix is used for evaluation because it can easily be represented in a heatmap and easily understood by ML engineers evaluating the performance of a model.

Accuracy is the percentage of total correct predictions by the model. It is the simplest metric since it provides a clear percentage of how often the model preticts the correct outcome.

Precision is how often a model correctly predicts a specific state compared to the total number of times it predicts that specific state, whether correctly or incorrectly. Precision is best used to avoid "False Positives" or predicting a state when it actually is a different one.

Recall is how often a model correctly predicts a specific state compared to the total number of times that state actually exists in the dataset. Recall is best used to avoid "False Negatives" or predicting that it is not a specific state when it actually is that specific state.

The F1 score is a combination of precision and recall such that:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

The performance metric most important for this problem is the F1 score which balances out the false positive and false negative states of precision and recall. In addition, because this is a dataset that includes three states rather than a binary positive or negative, giving a weighted score is more useful than a simple proportion.

In [59]:
```
# if you want to load other trained model, you have to change the model filename, export_name in this case
reloaded = tf.keras.models.load_model(export_name)
```

```
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics
` will be empty until you train or evaluate the model.
```

In [60]:
```
# let's call the summary of the model to see it is loaded correctly.
reloaded.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
reshape (Reshape)	(None, 128, 93, 1)	0
conv2d (Conv2D)	(None, 126, 91, 8)	80
max_pooling2d (MaxPooling2D)	(None, 42, 30, 8)	0
conv2d_1 (Conv2D)	(None, 40, 28, 16)	1,168
max_pooling2d_1 (MaxPooling2D)	(None, 13, 9, 16)	0
flatten (Flatten)	(None, 1872)	0
dense (Dense)	(None, 64)	119,872
dense_1 (Dense)	(None, 128)	8,320
output (Dense)	(None, 3)	387

Total params: 129,829 (507.15 KB)

Trainable params: 129,827 (507.14 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 2 (12.00 B)

In [61]:
```
# evaluate method will evaluate the model using the given data
# in this example, we wil use the test dataset splitted from the training data
# the output will be the loss (cross-entropy) and accuracy (metrics when compiling the model)
reloaded.evaluate(X_test, Y_test)
```

```
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 1.0000 - loss: 0.0347
```

Out[61]:
```
[0.03471129387617111, 1.0]
```

In [62]:
```
# predict method returns the result of the model by the given input data
# because we have 3 classes and the output node was trained by Softmax activation function,
# each data has 3 probabilities at the end
# sum of the 3 probabilities is 1.

Y_pred = reloaded.predict(X_test)
Y_pred
# is the result as you expected?
```

```
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step
```

Out[62]:
```
array([[7.18090689e-08, 8.15738797e-01, 1.84261173e-01],
       [9.97510433e-01, 1.10180849e-07, 2.48945621e-03],
       [1.03592915e-11, 9.97914851e-01, 2.08512042e-03],
       ...,
       [9.97507691e-01, 1.10255812e-07, 2.49217614e-03],
       [1.31408451e-02, 5.03963034e-04, 9.86355186e-01],
       [9.97509718e-01, 1.10165857e-07, 2.49018776e-03]], dtype=float32)
```

In [63]:
```
# Let's check the true output, Y_test in this case.
# As we did above, the principle axis means the class.
Y_test
```

Out[63]:
```
array([[0., 1., 0.],
       [1., 0., 0.],
       [0., 1., 0.],
       ...,
       [1., 0., 0.],
       [0., 0., 1.],
       [1., 0., 0.]])
```

In [64]:
```
# argmax method makes your prediction result to the label index
# among the 3 probabilities, element of th highest probability will be the principle axis for the label inde
# for example, [0 1 0] will be 1.

# run this and see if the result is as expected.

Y_pred_ind = Y_pred.argmax(1)
print("Prediction label index:", Y_pred_ind)
```

```
Prediction label index: [1 0 1 ... 0 2 0]
```

In [65]:
```
# We do the same for the true Y (Y_test_ind)

Y_test_ind = Y_test.argmax(1)
print("True label index:", Y_test_ind)
```

```
True label index: [1 0 1 ... 0 2 0]
```

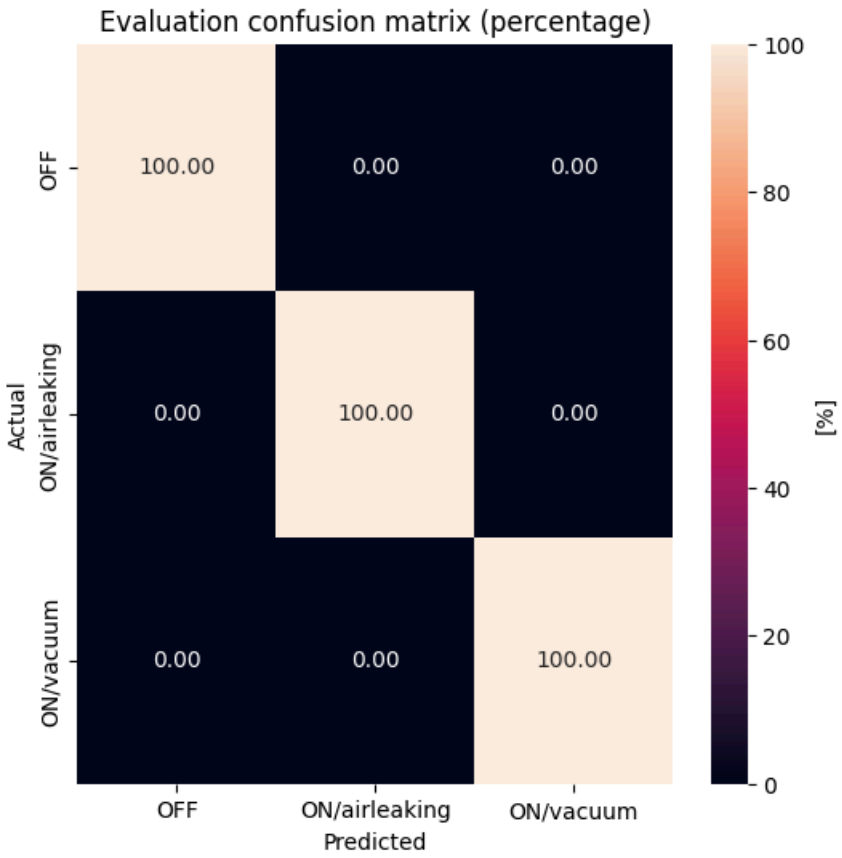
In [66]:
```
# using LabelEncoder object we defined,
# we decode the encoded index to the actual labels, 0:'OFF', 1:'ON/airleaking', 2:'ON/vacuum'

Y_test_label = enc.inverse_transform(Y_test_ind)
Y_pred_label = enc.inverse_transform(Y_pred_ind)
print("True labels:", Y_test_label)
print("Prediced labels:", Y_pred_label)
```

```
True labels: ['ON/airleaking' 'OFF' 'ON/airleaking' ... 'OFF' 'ON/vacuum' 'OFF']
Prediced labels: ['ON/airleaking' 'OFF' 'ON/airleaking' ... 'OFF' 'ON/vacuum' 'OFF']
```

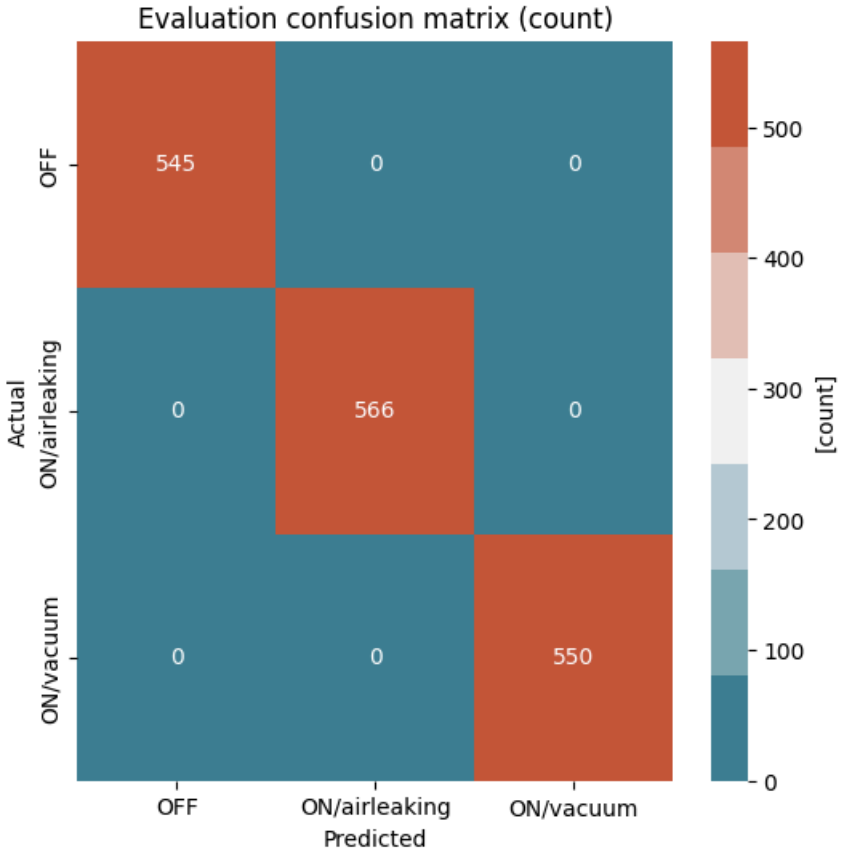
Basically, you performed basic evaluation (only loss and accuracy) and handling the prediction results. Let's try evaluating the model in detail using confusion matrix and other performance metrics.

In [67]:
```
# plot_cm function is for plotting confusion matrix

from sklearn.metrics import confusion_matrix


def plot_cm(y_true, y_pred, class_names, exp: str):
    fig, ax = plt.subplots(figsize=(6, 6))

    cm = confusion_matrix(y_true, y_pred)
    if exp == "percentage":
        cmn = cm.astype("float") / cm.sum(axis=1)[:, np.newaxis] * 100
        ax = sns.heatmap(
            cmn, annot=True, fmt=".2f", ax=ax, cbar_kws={"label": "[%]"}
        )
    else:
        cmn = cm
        ax = sns.heatmap(
            cmn,
            annot=True,
            fmt="d",
            cmap=sns.diverging_palette(220, 20, n=7),
            ax=ax,
            cbar_kws={"label": "[{}]".format(exp)},
        )

    plt.ylabel("Actual")
    plt.xlabel("Predicted")
    plt.title("Evaluation confusion matrix ({})".format(exp))
    ax.set_xticklabels(class_names)
    ax.set_yticklabels(class_names)
    b, t = plt.ylim()  # discover the values for bottom and top
    plt.ylim(b, t)  # update the ylim(bottom, top) values
    plt.show()
```

In [68]:
```
# Evaluation: Confusion matrix, percentage
plot_cm(Y_test_ind, Y_pred_ind, enc.classes_, "percentage")
```



In [69]:
```
# Evaluation: Confusion matrix, count
plot_cm(Y_test_ind, Y_pred_ind, enc.classes_, "count")
```



In [70]:
```
# performance metrics in micro average
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
)

pred_accuracy = accuracy_score(Y_test_ind, Y_pred_ind)
pred_precision = precision_score(Y_test_ind, Y_pred_ind, average="macro")
pred_recall = recall_score(Y_test_ind, Y_pred_ind, average="macro")
pred_f1 = f1_score(Y_test_ind, Y_pred_ind, average="macro")

print("Accuracy: {:.2f}%".format(pred_accuracy * 100))
print("Precision: {:.2f}%".format(pred_precision * 100))
print("Recall: {:.2f}%".format(pred_recall * 100))
print("F1-score: {:.2f}%".format(pred_f1 * 100))
```

```
Accuracy: 100.00%
Precision: 100.00%
Recall: 100.00%
F1-score: 100.00%
```

In [71]:
```
# performance metrics in each class

pred_precision_each = precision_score(Y_test_ind, Y_pred_ind, average=None)
pred_recall_each = recall_score(Y_test_ind, Y_pred_ind, average=None)
pred_f1_each = f1_score(Y_test_ind, Y_pred_ind, average=None)

precision_result_each = (
    enc.classes_[0],
    pred_precision_each[0] * 100,
    enc.classes_[1],
    pred_precision_each[1] * 100,
    enc.classes_[2],
    pred_precision_each[2] * 100,
)
recall_result_each = (
    enc.classes_[0],
    pred_recall_each[0] * 100,
    enc.classes_[1],
    pred_recall_each[1] * 100,
    enc.classes_[2],
    pred_recall_each[2] * 100,
)
f1_result_each = (
    enc.classes_[0],
    pred_f1_each[0] * 100,
    enc.classes_[1],
    pred_f1_each[1] * 100,
    enc.classes_[2],
    pred_f1_each[2] * 100,
)

print(
    "Precision each: {}={:.2f}%, {}={:.2f}%, {}={:.2f}%".format(
        *precision_result_each
    )
)
print(
    "Recall each: {}={:.2f}%, {}={:.2f}%, {}={:.2f}%".format(
        *recall_result_each
    )
)
print("F1 each: {}={:.2f}%, {}={:.2f}%, {}={:.2f}%".format(*f1_result_each))
```

```
Precision each: OFF=100.00%, ON/airleaking=100.00%, ON/vacuum=100.00%
Recall each: OFF=100.00%, ON/airleaking=100.00%, ON/vacuum=100.00%
F1 each: OFF=100.00%, ON/airleaking=100.00%, ON/vacuum=100.00%
```

Task 2.5

Using the given examples above, try training different CNN architectures and select the best model. You may try changing number of CNN layers, number of hidden layers, hyperparameters, and so on. But you should take reasonable changes.

But, you can use the same feature (Mel-spectrogram, shape=(128,93)) above. Then **summarize your model (details of architecture), and evaluate it. Note that you need to use this model for next section and Lab10.**

New Architecture

For the new architecture I am training a model with:

1. Batch size of 300 (slightly higher than previous)
2. 10 Epochs (Double previous)
3. Learning rate of 10e-4 (same as previous)
4. Two CNN layers (same as previous)
5. Three hidden layers with 32, 64 and 128 neurons respectively (new)

In [92]:
```
## add more code blocks below as much as you need.
## Other hyperparameters
BATCH = 300  # the number of samples per gradient update
EPOCH = 10  # how many cycles will be trained in an entire dataset passed forward and backward through neura
LR = 10e-4  # learning rate is to control how much to change the model in response to the estimated error pe

model = tf.keras.Sequential()  # model obj. takes sequential model
model.add(
    layers.Reshape(
        (X_train.shape[1], X_train.shape[2], 1),
        input_shape=(X_train.shape[1], X_train.shape[2]),
    )
)  # reshaping the input to ingest 2D CNN layer
model.add(
    layers.Conv2D(filters=8, kernel_size=(3, 3), activation="relu")
)  # first 2D CNN layer
model.add(layers.MaxPooling2D(pool_size=(3, 3)))  # max pooling layer

model.add(
    layers.Conv2D(filters=16, kernel_size=(3, 3), activation="relu")
)  # second 2D CNN layer
model.add(layers.MaxPooling2D(pool_size=(3, 3)))  # max pooling layer

model.add(layers.Flatten())  # flatten layer

model.add(layers.Dense(32, activation="relu"))  # first hidden layer

model.add(layers.Dense(64, activation="relu"))  # second hidden layer

model.add(layers.Dense(128, activation="relu"))  # third hidden layer

model.add(
    layers.Dense(units=Y_train.shape[1], activation="softmax", name="output")
)  # output layer (node)

model.summary()  # This will show the model object

# compiling the CNN model, loss function is categorical crossentropy and training metric is accuracy
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=LR),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)

# This will start model training and take a few minutes
history = model.fit(
    X_train,
```

```
    Y_train,
    validation_data=(X_test, Y_test),
    batch_size=BATCH,
    epochs=EPOCH,
    verbose=1,
)

Y_pred = model.predict(X_test)
Y_pred_ind = Y_pred.argmax(1)
Y_test_ind = Y_test.argmax(1)

# performance metrics in micro average
pred_accuracy = accuracy_score(Y_test_ind, Y_pred_ind)
pred_precision = precision_score(Y_test_ind, Y_pred_ind, average="macro")
pred_recall = recall_score(Y_test_ind, Y_pred_ind, average="macro")
pred_f1 = f1_score(Y_test_ind, Y_pred_ind, average="macro")

print("Accuracy: {:.2f}%".format(pred_accuracy * 100))
print("Precision: {:.2f}%".format(pred_precision * 100))
print("Recall: {:.2f}%".format(pred_recall * 100))
print("F1-score: {:.2f}%".format(pred_f1 * 100))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/reshape.py:38: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
```

Model: "sequential_5"

Layer (type)	Output Shape	Param #
reshape_5 (Reshape)	(None, 128, 93, 1)	0
conv2d_10 (Conv2D)	(None, 126, 91, 8)	80
max_pooling2d_10 (MaxPooling2D)	(None, 42, 30, 8)	0
conv2d_11 (Conv2D)	(None, 40, 28, 16)	1,168
max_pooling2d_11 (MaxPooling2D)	(None, 13, 9, 16)	0
flatten_5 (Flatten)	(None, 1872)	0
dense_12 (Dense)	(None, 32)	59,936
dense_13 (Dense)	(None, 64)	2,112
dense_14 (Dense)	(None, 128)	8,320
output (Dense)	(None, 3)	387

Total params: 72,003 (281.26 KB)

Trainable params: 72,003 (281.26 KB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 10s 240ms/step - accuracy: 0.6869 - loss: 0.9881 - val_accuracy: 0.8332 - val_los
s: 0.6350
Epoch 2/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 26ms/step - accuracy: 0.8176 - loss: 0.3960 - val_accuracy: 0.8098 - val_loss:
0.3404
Epoch 3/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 28ms/step - accuracy: 0.8275 - loss: 0.3019 - val_accuracy: 0.8688 - val_loss:
0.2593
Epoch 4/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 25ms/step - accuracy: 0.8519 - loss: 0.2596 - val_accuracy: 0.8760 - val_loss:
0.2390
Epoch 5/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 28ms/step - accuracy: 0.8686 - loss: 0.2359 - val_accuracy: 0.9085 - val_loss:
0.2094
Epoch 6/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 25ms/step - accuracy: 0.9094 - loss: 0.1957 - val_accuracy: 0.9073 - val_loss:
0.1685
Epoch 7/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 25ms/step - accuracy: 0.9452 - loss: 0.1437 - val_accuracy: 0.9621 - val_loss:
0.1020
Epoch 8/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 28ms/step - accuracy: 0.9830 - loss: 0.0764 - val_accuracy: 0.9952 - val_loss:
0.0413
Epoch 9/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 25ms/step - accuracy: 0.9986 - loss: 0.0305 - val_accuracy: 1.0000 - val_loss:
0.0165
Epoch 10/10
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 26ms/step - accuracy: 1.0000 - loss: 0.0132 - val_accuracy: 1.0000 - val_loss:
0.0071
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step
Accuracy: 100.00%
Precision: 100.00%
Recall: 100.00%
F1-score: 100.00%
```

In [93]:

```python
import datetime

t = datetime.datetime.now().strftime(
    "%Y%m%d_%H%M%S"
)  # generate date time string to identify the model generated time
export_name = t + "_Prelab10_CNN_model_Best.h5"  # this is eventually filename.
model.save(export_name)  # save method save the model with the file name
print("Model was exported in this path: '{}'".format(export_name))
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)
`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.sav
e('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
Model was exported in this path: '20260409_024736_Prelab10_CNN_model_Best.h5'
```

Please continue to Prelab 10.3 here.